

PRODUCT REQUIREMENTS & TECHNICAL PROPOSAL

CRM

Car · Hotel · Flight Booking Management Platform

Secure, Role-Based, Audit-Ready Booking & Lead Management

| Document             | Details                                                                        |
|----------------------|--------------------------------------------------------------------------------|
| Prepared For         | Client                                                                         |
| Scope                | Lead-to-Booking Workflow, RBAC & Security, Database Schema, Technology Options |
| Modules Covered      | Car Rental, Hotel, Flight Bookings                                             |
| Architecture Options | Option A: Full-Stack Next.js (TypeScript)   Option B: Next.js + FastAPI        |

# 1. Executive Summary

---

This document presents the complete product and technical blueprint for a unified, high-security Travel CRM built to manage the full lifecycle of Car Rental, Hotel, and Flight bookings — from first customer contact through payment authorization, billing, quality control, and post-booking servicing.

The platform is built around three core principles:

- **Lead-First Data Integrity** — agents create the Lead directly from the customer's Name, Number, and Email, and the system automatically runs a duplicate-detection check at that point, eliminating orphaned or duplicated customer records.
- **Status-Driven Departmental Workflow** — every booking moves through a defined set of statuses (Authorization Pending → Client Approved → Transferred to Billing → Card Charged/Declined → Departmental Tagging → QC Done), and visibility plus notifications are automatically computed from the current status.
- **Security & Compliance by Design** — role-based access control, IP whitelisting, default data masking of PII (email, phone, card details), and an immutable audit trail for every sensitive action, including a full Reveal/Access log restricted to Admin oversight.

This proposal includes the complete departmental workflow, the underlying relational database schema, the security and RBAC model, and two viable technology stack options — a single-language full-stack Next.js/TypeScript build, and a Next.js + FastAPI hybrid — so the client can choose the architecture that best matches their team and long-term roadmap.

## 2. Project Overview & Objectives

---

The goal is to build a high-security, role-based CRM that streamlines the booking lifecycle — from lead acquisition to payment authorization and quality control — while ensuring every action is logged and every sensitive data field is protected.

### 2.1 Core Objectives

- Enforce a single, verified entry point for every new customer interaction across all three service lines.
- Provide agents, billing, team leads, auditors, and support staff with a workspace scoped to exactly what their role needs to see.
- Automate the handoff between departments using status changes as workflow triggers, with real-time push notifications.
- Protect customer PII (email, phone, card data) with masking-by-default and a fully logged reveal mechanism.
- Maintain a real-time mirrored copy of key data in Google Sheets for external reporting.
- Support modification, cancellation, refund, and future-credit workflows without losing the original booking history.

## 3. User Roles & Access Control (RBAC)

---

Access to the CRM is strictly role-based. There is no public self-registration — all users are provisioned manually by an Admin or Super Admin, and Super Admin retains a global toggle to enable or disable registration entirely.

### 3.1 Defined Roles

| Role                  | Primary Responsibility            | Key Permissions / Triggers                                                                                  |
|-----------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------------|
| Super Admin           | Global oversight & provisioning   | Full visibility across all leads; manages registration toggle; grants custom access; views master audit log |
| Admin                 | System administration             | Provisions users, assigns roles; access to full “Log Report of Booking Process”                             |
| Agent                 | Lead entry & customer interaction | Creates Leads and Bookings directly; sees only their own leads by default                                   |
| Billing               | Payment processing                | Processes charges; triggers Card Charged / Card Declined status                                             |
| TL (Team Lead)        | Full departmental oversight       | Full visibility; exclusive rights to create Future Credits                                                  |
| Auditor               | Quality control                   | Exclusive rights to trigger “QC Done” status; cross-service audit visibility                                |
| CS (Customer Service) | Post-booking support              | Creates Future Credits; supports modification/cancellation requests                                         |
| Change Dep            | Reservation modifications         | Handles the Original vs. Revised modification workflow                                                      |
| Chargeback Dep        | Dispute management                | Manages RDR and chargeback tagging and resolution                                                           |
| CR Booking            | Car rental fulfillment            | Confirms and fulfills car rental reservations                                                               |

### 3.2 Access Rules

- IP Whitelisting: access is restricted to approved IP addresses per user; enforced at the database/session layer.
- Ownership Model: agents see only the leads they created.
- Status-Based Sharing: visibility automatically expands to the relevant department the moment a lead's status changes (e.g., moving to “Transferred to Billing” grants the Billing role access).
- Global View: Super Admins can view all leads across all departments and can manually grant ad-hoc access to specific records or teams.
- Access Notifications: opening any existing CRM record notifies both the Admin and the original creating Agent.

## 4. Lead Creation & Verification Workflow

---

Every new customer interaction — regardless of whether it becomes a Car, Hotel, or Flight booking — begins with the agent creating a Lead directly, with an automatic verification check before the full booking form unlocks.

### 4.1 The “Lead-First” Flow

- **Step 1 — Lead Entry:** the agent creates a new Lead by entering the customer's Name, Number, and Email directly — this is the single entry point into the CRM, with no separate intake step required.
- **Step 2 — Automatic Duplicate Search:** on submission, the system automatically searches existing Leads and Booking records for a match on Name, Number, OR Email.
- **Step 3 — Conditional Prompt:** if a match is found, the UI displays “Client already existed — do you still want to proceed?” The agent must explicitly confirm before continuing.
- **Step 4 — Unlock:** once verification clears (or is explicitly overridden), the full Booking Form becomes visible and the agent selects the service type — Car, Hotel, or Flight — which dynamically renders the correct fields.

#### Why this matters

Running duplicate detection at the moment of Lead creation prevents database pollution and split records, guaranteeing a clean, de-duplicated record from the very first customer touchpoint — without adding an extra step for agents.

## 5. Service-Specific Booking Modules

---

All three service lines share the same lead, security, and workflow engine. Only the booking-detail fields differ.

### 5.1 Car Rental Module

- Booking Reference, Booking Platform (e.g., eBookingHub, Our Booking), Car Provider (Hertz, Budget, etc.)
- Renter Name & Date of Birth, Transmission (Automatic/Manual), Fuel Policy
- Pick-up Date/Time & Location, Return Date/Time & Location
- **Vehicle Type Dropdown (standardized):** Economy, Compact, Intermediate, Standard, Full Size, Standard SUV, Intermediate SUV, Premium SUV, Full-Size SUV, Luxury, Passenger Van, Mini Van, 15-Passenger Van, Mystery Car, Premium Crossover, Premium Elite Crossover, Pickup Truck, Electric.
- Payment Breakdown: Prepaid Amount (charged by agency) vs. Pay-at-Counter Amount (paid directly to provider) vs. Total Amount.

### 5.2 Hotel Module

- Search filters consistent with the Car module: Date Range and Location.

- Hotel Name, Room Type, and Stay Duration.
- Same prepaid / pay-at-counter payment breakdown pattern as Car Rental.

### 5.3 Flight Module

- PNR (Passenger Name Record) and Airline.
- Flight Number(s) and Route (Origin / Destination).
- Cabin Class.

#### Unified Dashboard

Regardless of service type, all leads share consistent status badges (e.g., “Client Approved,” “QC Done”) and the same search filters (Date, Email, Mobile), so Billing and Audit teams manage every queue without switching contexts.

## 6. Booking Lifecycle & Status Workflow

Every booking is driven by a single `current_status` field. Status transitions act as a state machine: they simultaneously change who can see the record and fire a push notification to the next responsible department.

### 6.1 Status Reference Table

| Status                 | Set By                       | UI Colour  | Notifies                |
|------------------------|------------------------------|------------|-------------------------|
| Authorization Pending  | System (default)             | Grey       | — (default state)       |
| Client Approved        | Customer (via “I Authorize”) | Blue       | Admin                   |
| Transferred to Billing | Agent                        | Purple     | Billing                 |
| Card Charged           | Billing                      | Green      | Agent                   |
| Card Declined          | Billing                      | Red        | Agent                   |
| Tag to Change Dep      | Any authorized role          | Yellow     | Change Dep              |
| Tag to CR Booking      | Any authorized role          | Orange     | CR Booking              |
| Tag to Auditor         | Any authorized role          | Cyan       | Auditor, Admin          |
| QC Done                | Auditor                      | Dark Green | Agent, Admin            |
| Tag Refund             | Billing                      | Red        | Billing                 |
| Tag RDR                | Billing                      | Black      | Billing, Chargeback Dep |
| Tag Chargeback         | Billing                      | Black      | Billing, Chargeback Dep |

## 6.2 Standard Flow

Agent Creates Lead → New Booking Form → Authorization Pending → Customer clicks “I Authorize” → Client Approved → Transferred to Billing → Card Charged (or Declined) → Departmental Tagging (Change Dep / CR Booking / Auditor / Chargeback Dep as needed) → QC Done.

Automatic notification triggers fire on every transition, so the responsible department is alerted the moment a case reaches their stage — no manual handoff required.

## 7. Modification, Cancellation & Future Credit Management

---

### 7.1 Modifications — Original vs. Revised

Every change to a live booking is captured as a paired snapshot rather than an in-place overwrite, preserving full history for audit purposes.

- Tracked fields: Booking Reference, Vehicle/Room/Flight, Pick-up/Return or Stay/Route details, Duration.
- The system automatically compares Original vs. Revised values to calculate any Modification Amount owed or refunded.

### 7.2 Cancellations & Refunds

- Original Prepaid Amount is compared against the Cancellation Penalty Fee.
- System calculates the Refund Amount (returned to customer) and the Final Retained Amount (kept by the agency).

### 7.3 Future Credits

- Creation restricted exclusively to TL and CS roles.
- Visible (read-only) to Billing, CS, Change Dep, Chargeback Dep, and Auditor once issued.
- Tracked fields: source CRM ID, Voucher Amount, Number of Vouchers, Validity Date, and Created By.

## 8. Payment Authorization & Electronic Consent

---

To prevent chargebacks and disputes, every booking generates a dynamic customer-facing authorization form before any charge is processed.

### 8.1 Payment Breakdown

- Prepaid Amount — the amount charged directly by the agency at time of booking.
- Pay at Counter — the remaining balance payable to the provider on arrival/pickup.
- Total Amount — the sum of both, always shown to the customer for full transparency.

## 8.2 “I Authorize” Consent Capture

When the customer clicks “I Authorize,” the system records a full consent package tied to that specific CRM record:

- Cardholder confirmation (“I am the authorized holder of the card”)
- Authorization to charge the Prepaid Amount, and acknowledgement of the Pay-at-Counter amount
- Review/approval of booking details and authorization to complete the reservation
- Acknowledgement that rental/hotel/airline terms apply and that the charge is non-refundable / non-disputable
- Agreement to all terms and conditions
- Metadata: exact Timestamp, Customer IP Address, Browser/Device information, and Consent Status

## 9. Security, Data Masking & Audit Logging

---

### 9.1 Data Masking (“Click-to-Reveal”)

Email, Phone Number, and Credit Card details are masked by default in every view (e.g., exa\*\*\*@mail.com, \*\*\*\*\_\*\*\*\*\_\*\*\*\*-1234). An agent must explicitly click “Reveal” to see the raw value, and that action is sent to the backend as a discrete, logged request — raw values are never shipped to the client by default.

### 9.2 Reveal / PII Access Audit Log

Every reveal action is captured for Super Admin / Admin review and includes:

- Agent ID and Name
- Field revealed (Email, Phone, or Card)
- Timestamp, IP Address, and Browser/Device info
- The CRM ID being accessed, and the agent-provided reason for access

### 9.3 Booking Process Log (Admin-only)

A comprehensive, immutable lifecycle log of every Booking\_Core change is maintained and restricted strictly to the Admin role — this is the master “Log Report of the Booking Process” referenced throughout this document.

### 9.4 IP Whitelisting

Database-level checks verify that any user attempting an update has an active session originating from a whitelisted IP address whenever IP whitelisting is enabled for that user.

## 10. Notifications, Real-Time Sync & External Integration

---

### 10.1 Notifications

- Push notifications fire automatically for every status change, routed to the department mapped in the Status Reference Table.
- Opening any existing CRM record notifies both the Admin and the original owning Agent.

### 10.2 Real-Time Google Sheets Sync

A queuing/webhook mechanism mirrors key tables (starting with Booking\_Core) to an external Google Sheet in real time, giving stakeholders a live, always-current reporting view outside the CRM itself, with sync status tracked (Pending / Success / Failed) for reliability.

### 10.3 Modular External Integration

A standardized mapping layer exposes a unified capture endpoint (e.g., /api/leads/capture) so that data from external booking engines, website forms, or third-party provider APIs can be transformed into the CRM's internal schema without touching core booking logic — new integrations (e.g., a car provider's live vehicle-type feed) can be added without a rewrite.

## 11. Database Schema Summary

---

The schema is organized around a central Lead/CRM record — created directly by the Agent — that links to a service-specific booking module, with dedicated tables for security, payments, and lifecycle history. Field-level constraints (data types, primary/foreign keys) are detailed in the accompanying technical schema appendix; the structure below summarizes the core entities and relationships.

### 11.1 Core Entity Overview

| Table                        | Purpose                                                                                      | Key Relationship             |
|------------------------------|----------------------------------------------------------------------------------------------|------------------------------|
| Users / Roles                | Identity, role assignment, IP whitelisting                                                   | 1 User → many Leads          |
| Leads / Booking_Core         | Central CRM record: Name/Number/Email, status, service type, ownership, duplicate-check flag | 1 Lead → 1 Service Booking   |
| Car / Hotel / Flight Booking | Service-specific reservation details                                                         | belongs to one Lead (crm_id) |
| Booking_Modifications        | Original vs. Revised change tracking                                                         | many per Booking             |
| Cancellations                | Refund / penalty / retained-amount calculation                                               | 1 per cancelled Booking      |
| Payment_Transactions         | Prepaid / Pay-at-Counter / Total breakdown                                                   | 1 per Booking                |
| Future_Credits               | TL/CS-issued vouchers                                                                        | linked to source crm_id      |
| Authorization_Records        | “I Authorize” consent capture                                                                | 1 per authorization event    |

|                           |                                             |                     |
|---------------------------|---------------------------------------------|---------------------|
| Status_Lookup             | Status names, colours, notification routing | referenced by Leads |
| Access_Notification_Log   | Record-open notifications                   | per view event      |
| PII_Reveal_Audit_Log      | Click-to-reveal audit trail                 | per reveal event    |
| Google_Sheets_Sync_Status | External sync tracking                      | per synced record   |

## 11.2 Relationship Flow

- Agent creates a Lead directly (Name, Number, Email) → automatic duplicate check runs against existing Leads/Bookings → Lead (crm\_id) is confirmed and the booking form unlocks.
- The Lead links to exactly one Car, Hotel, or Flight booking record.
- Once the customer clicks “I Authorize,” an Authorization Record is linked to that Lead.
- Every status change updates the Lead and triggers a notification to the next department, per the Status\_Lookup mapping.
- Visibility is computed dynamically: filtered by agent\_id (creator-only) or expanded by current\_status (department access).

A full field-level technical schema (data types, constraints, and lookup values for every table above) is available as a companion appendix and can be handed directly to the engineering team implementing either technology option below.

## 12. Technology Stack Options

Two architecture options are presented below. Both fully satisfy every requirement in this document — the RBAC model, the Lead-First gate, status-driven workflow, data masking, audit logging, and real-time notifications/sync. The choice is primarily about team composition, hiring pool, and long-term operational preference rather than a difference in capability.

### 12.1 Option A — Full-Stack Next.js (Next.js + TypeScript End-to-End)

A single-language architecture where Next.js handles both the frontend (React/App Router) and the backend, via Next.js Route Handlers / Server Actions, all written in TypeScript.

| Layer           | Technology                                                                                                   |
|-----------------|--------------------------------------------------------------------------------------------------------------|
| Frontend        | Next.js (App Router) + React + TypeScript                                                                    |
| Backend / API   | Next.js Route Handlers & Server Actions (TypeScript) — same codebase and deploy unit as the frontend         |
| Database        | PostgreSQL, accessed via Prisma or Drizzle ORM (type-safe queries end-to-end)                                |
| Auth & Sessions | NextAuth.js (Auth.js) for session/role/IP-aware auth                                                         |
| Real-time       | WebSockets (e.g., Pusher / Socket.io) or a real-time DB listener for live notifications and dashboard counts |

|                 |                                                                               |
|-----------------|-------------------------------------------------------------------------------|
| Background Jobs | Queue (e.g., BullMQ + Redis) for Google Sheets sync and notification dispatch |
| Hosting         | Vercel, or any Node-compatible host                                           |

- **Advantages:** one language (TypeScript) across the entire stack, shared types between client and server, a single deployable unit, faster iteration for small-to-mid teams, and a large hiring pool of full-stack JS/TS engineers.
- **Trade-offs:** very heavy background processing (e.g., large-scale data pipelines, ML-driven fraud scoring) is possible but less natural in a Node runtime than in a dedicated Python service.

## 12.2 Option B — Next.js Frontend + FastAPI Backend

A polyglot architecture where Next.js/TypeScript owns the entire UI layer, and a separate FastAPI (Python) service owns all business logic, database access, and background processing, communicating over a versioned REST/JSON API.

| Layer           | Technology                                                                                                     |
|-----------------|----------------------------------------------------------------------------------------------------------------|
| Frontend        | Next.js (App Router) + React + TypeScript                                                                      |
| Backend / API   | FastAPI (Python) exposing a versioned REST API consumed by Next.js                                             |
| Database        | PostgreSQL, accessed via SQLAlchemy + Alembic migrations, with Pydantic models for request/response validation |
| Auth & Sessions | OAuth2/JWT issued by FastAPI, validated on both sides; role & IP checks enforced server-side in Python         |
| Real-time       | FastAPI WebSocket routes, or a message broker (Redis Pub/Sub) feeding Next.js clients                          |
| Background Jobs | Celery (or FastAPI background tasks) + Redis for Google Sheets sync, audit processing, and scheduled tasks     |
| Hosting         | Next.js on Vercel; FastAPI on a container platform (e.g., AWS/GCP, Render, Fly.io)                             |

- **Advantages:** clean separation of concerns between UI and business logic; Python's mature ecosystem is a strong fit if future requirements involve data science, ML-based fraud/chargeback prediction, or heavy document/report processing; FastAPI's automatic OpenAPI docs simplify onboarding a larger or distributed engineering team.
- **Trade-offs:** two codebases and two deploy pipelines to maintain; requires engineers comfortable in both TypeScript and Python; slightly more infrastructure to stand up initially.

## 12.3 Side-by-Side Comparison

| Criteria                       | Option A: Full-Stack Next.js   | Option B: Next.js + FastAPI        |
|--------------------------------|--------------------------------|------------------------------------|
| Languages                      | TypeScript only                | TypeScript (front) + Python (back) |
| Team size fit                  | Small – mid teams              | Mid – large / distributed teams    |
| Time to first release          | Faster (single codebase)       | Slightly longer (two services)     |
| Best for                       | Rapid delivery, unified team   | Complex logic, future ML/data work |
| Type safety                    | End-to-end via shared TS types | Strong per-side (TS + Pydantic)    |
| Background/async processing    | Good (Node queues)             | Excellent (Python ecosystem)       |
| Ops complexity                 | Lower (one deploy target)      | Higher (two deploy targets)        |
| RBAC / masking / audit support | Fully supported                | Fully supported                    |

### Recommendation

Both options meet 100% of the functional and security requirements in this document. If the priority is the fastest path to a production-ready v1 with a lean team, Option A (Full-Stack Next.js) is the more efficient path. If the roadmap includes heavier data processing, fraud/ML scoring, or a larger backend team already working in Python, Option B (Next.js + FastAPI) is the stronger long-term foundation.

## 13. Next Steps

---

- Confirm the preferred technology option (A or B) and finalize the deployment environment.
- Review and sign off on the RBAC role list and Status Reference Table.
- Finalize the field-level technical database schema appendix for engineering hand-off.
- Approve the deployment timeline and begin the build.